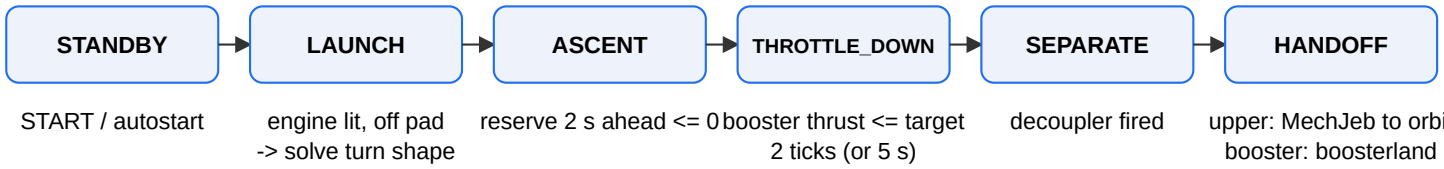
ascent: how it works

Flies a two-stage stack from the pad: MechJeb's **classic** ascent steers and throttles; this program chooses the turn shape for the craft, drops the booster at the last moment it can still fly home, and follows the **upper stage** to a circular orbit (`ORBIT_ALT_M`, 80 km). The booster is flown afterwards from FMRS's separation save by `boosterland/fmrs.py` (or, with `HANDOFF=booster`, in this process by boosterland, the upper stage abandoned).

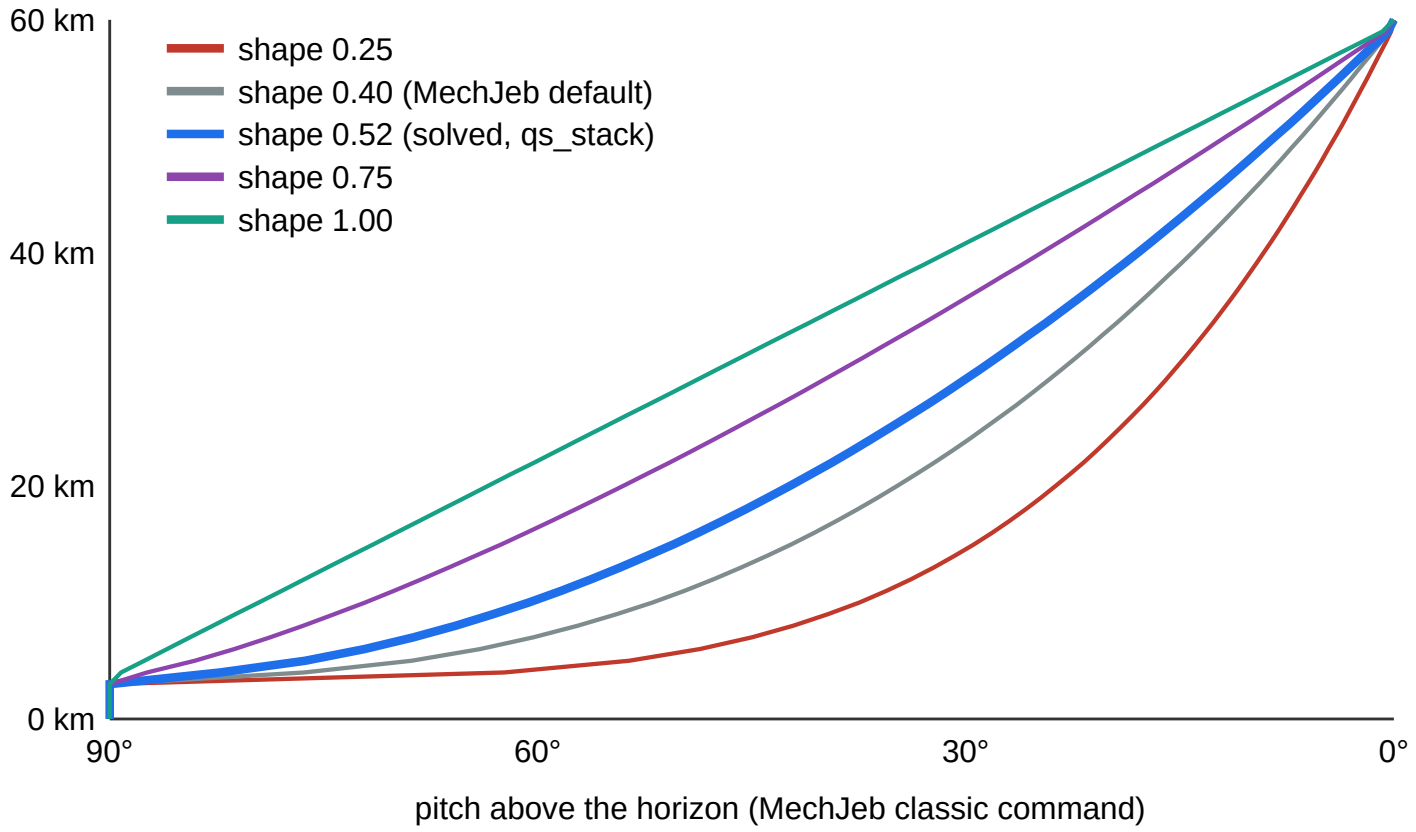
Phases



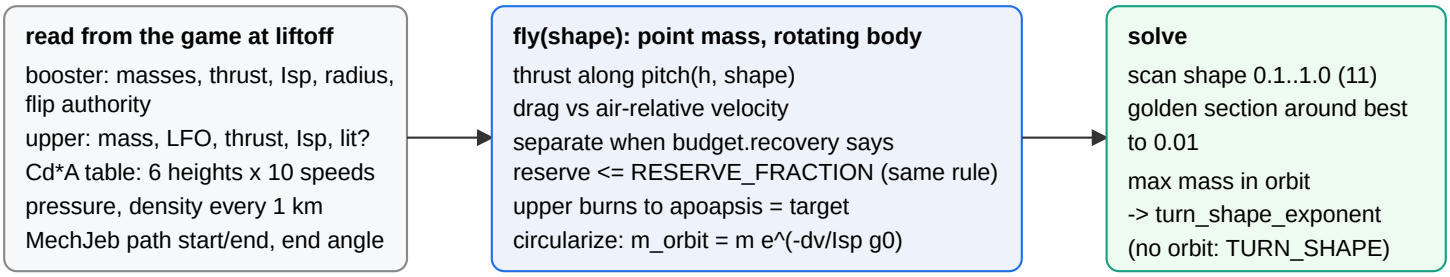
phase	what it does	leaves when
STANDBY	sets MechJeb: classic, 80 km, inclination, turn shape fallback, roll = the pad roll against a heading-90 pitch-over (no roll on an eastward launch); MechJeb autostage off; engages the ascent	START pressed (or autostart)
LAUNCH	stages until an engine is *active* and the vessel is off the pad; never fires a stage that decouples anything	lit and not landed / pre-launch: builds the `Stack` (booster = the parts the next decoupling drops that carry lit engines), the booster model, **solves the turn shape**
ASCENT	every tick: `budget.recovery` -- propellant the booster would land with if it separated now (boostback with the flip, ballistic return, landing burn from terminal velocity); reserve = left - `RESERVE_FRACTION` (0.15) x need, projected `SEP_LEAD_S` ahead at its rate of fall	above `MIN_ALT_M` and the projected reserve <= 0, or the booster is dry
THROTTLE_DOWN	booster engine thrust limits (outer engines first) so its free acceleration is `SEP_ACCEL_RATIO` 0.75 of the upper stage's; MechJeb keeps the main throttle	delivered booster thrust under target on 2 ticks, or 5 s
SEPARATE	fires the booster's decoupler, sets the booster's throttle to 0; its limits stay on	at once
HANDOFF (upper)	MechJeb autostage on; MechJeb burns to the apoapsis, coasts, circularizes	MechJeb disengages (logs orbit and mass), the upper stage is lost (mass 0 / landed / splashed), or `ORBIT_TIMEOUT_S`

The turn shape

MechJeb classic commands pitch as a function of altitude only: pitch = 90° below the turn start, the end angle above the turn end, and between them $90 - ((h - start)/(end - start))^{shape} \times (90 - end\ angle)$. Start and end come from MechJeb's automatic path (on Kerbin 3.5 km and 59.5 km). A low shape turns early and flat -- the booster ends up lower and further downrange; a high shape climbs steeply and turns late.



A flat turn costs drag and sends the booster further out, so it must keep more propellant for the trip back and separates earlier; a steep turn loses to gravity. Because separation is decided by the booster's own return budget, both costs show up in one number: **the upper stage's mass in the target orbit**. `turnshape.solve` maximizes it.



The model flies a point mass in the equatorial plane of the rotating body: thrust along the commanded pitch with Isp and thrust interpolated by pressure (constant mass flow), drag from the craft's own Cd*A table sampled from the game at zero angle of attack, separation by exactly the rule ASCENT uses, the upper stage to the apoapsis target and the circularization at its vacuum Isp. Left out: angle of attack, the booster throttle-down, drag in the coast. The solve takes about half a second; its result and the predicted curve are logged as `turn shape: solved ...`.

Rules the code keeps

- The throttle is MechJeb's: the program never writes or reads `control.throttle` on the ascending vessel (kRPC would hold a written value over MechJeb); the throttle-down reads the booster engines' delivered/allowed thrust.
- The booster keeps its throttle-down limits after separation; whoever flies it next owns them (boosterland's cluster for several engine groups; `fmrs.py` clears them for one).
- Every tunable is in `ascent/config.py`; every log carries the config differences and the defaults fingerprint.